Security, Governance & Monitoring

> **Series:** WFLOW | **Notebook:** 9 of 9 | **Created:** January 2026 | **Last Updated:** 01/28/2026

Production Best Practices

This final notebook covers workflow security, governance, observability, and operational best practices for running workflows in production.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	`automation:workflows:admin` for governance setup
Prior Knowledge	**WFLOW-01** through **WFLOW-08**

1. Security Best Practices

Secure Workflow Design

Principle	Implementation
Least privilege	Connections with minimum required permissions
No hardcoded secrets	Use secrets vault, never inline credentials
Input validation	Sanitize all dynamic inputs
Audit logging	Log all actions for compliance
Fail secure	Default to safe state on errors

Input Sanitization

```
export default async function({ event }) {
  // NEVER do this - injection risk
  // const query = `fetch logs | filter content == "${event().title}"`;

  // SAFE: Validate and sanitize inputs
  const title = event().title;

  // Remove potentially dangerous characters
  const sanitized = title
    .replace(/["'\\]/g, '')  // Remove quotes and backslashes
    .substring(0, 200);        // Limit length

  // Use parameterized queries when possible
  return { sanitized_title: sanitized };
}
```

Secure HTTP Requests

```
export default async function({ event, env }) {
  // NEVER log or expose secrets
  // console.log(env.API_TOKEN);  // BAD!

  // ALWAYS use HTTPS
  const response = await fetch('https://api.example.com/endpoint', {  // NOT
http://
    headers: {
      'Authorization': `Bearer ${env.API_TOKEN}`  // Token from secrets
    }
  });

  // DON'T return sensitive data
  return {
    status: response.status,
    success: response.ok
    // NOT: response_body: await response.text()  // May contain secrets
  };
}
```

2. Secrets Management

Using Environment Secrets

Secrets are stored securely and accessed via `env.SECRET_NAME`.

Creating Secrets:

1. Open workflow editor
2. Go to **Settings** → **Secrets**
3. Add secret with name and value
4. Reference as `{{ env.SECRET_NAME }}`

Secret Naming Convention

Pattern	Example	Use
`<SERVICE>_API_TOKEN`	`SLACK_API_TOKEN`	API authentication
`<SERVICE>_WEBHOOK_URL`	`PAGERDUTY_WEBHOOK_URL`	Webhook endpoints
`<SERVICE>_<ENV>_CRED`	`SERVICENOW_PROD_CRED`	Environment-specific

Secrets in JavaScript

```
export default async function({ env }) {
  // Access secrets via env object
  const apiToken = env.EXTERNAL_API_TOKEN;
  const webhookUrl = env.SLACK_WEBHOOK_URL;

  // Secrets are never logged or exposed
  if (!apiToken) {
    throw new Error('Missing required secret: EXTERNAL_API_TOKEN');
  }

  return { configured: true };
}
```

Secrets in YAML

```
input:
  url: "https://api.example.com/webhook"
  headers:
    Authorization: "Bearer {{ env.API_TOKEN }}"
    X-Api-Key: "{{ env.API_KEY }}"
```

Secret Rotation

1. Create new secret with updated value
2. Update workflow to use new secret
3. Test workflow execution
4. Remove old secret

3. Access Control (RBAC)

Workflow Permissions

Permission	Allows
`automation:workflows:read`	View workflows, view execution history
`automation:workflows:write`	Create, edit, enable/disable workflows
`automation:workflows:run`	Execute workflows manually
`automation:workflows:delete`	Delete workflows
`automation:workflows:admin`	Full admin including secrets
`automation:connections:read`	View connections
`automation:connections:write`	Create, edit connections

IAM Policy Example

```
ALLOW automation:workflows:read;
ALLOW automation:workflows:write WHERE workflow.owner == "${user.email}";
ALLOW automation:workflows:run;
```

Team-Based Access Pattern

Team	Permissions
Workflow Admins	Full admin access
SRE Team	Read, write, run all workflows
App Teams	Read, write, run own workflows
Viewers	Read-only access

Workflow Ownership

```
# Include ownership metadata in workflow
metadata:
  owner: sre-team@company.com
  team: platform
  classification: production
```

4. Workflow Observability

Key Metrics to Monitor

Metric	Query	Alert Threshold
Execution success rate	See cell below	< 95%
Execution duration	See cell below	> 5 min avg
Failed executions	See cell below	> 5 per hour
Rate limit hits	See cell below	Any

Execution History Dashboard

Build a dashboard with these queries to monitor workflow health.

```
// Overall workflow health — last 24 hours
fetch events, from: now() - 24h
| filter event.type == "automation.workflow.execution"
| summarize
    total = count(),
    succeeded = countIf(execution.status == "SUCCEEDED"),
    failed = countIf(execution.status == "FAILED"),
    timed_out = countIf(execution.status == "TIMED_OUT")
| fieldsAdd success_rate = round(100.0 * succeeded / total, decimals: 2)
```

```
// Per-workflow success rates
fetch events, from: now() - 7d
| filter event.type == "automation.workflow.execution"
| summarize
    total = count(),
    succeeded = countIf(execution.status == "SUCCEEDED"),
    failed = countIf(execution.status == "FAILED"),
    avg_duration = avg(execution.duration),
    by:{workflow.name}
| fieldsAdd success_rate = round(100.0 * succeeded / total, decimals: 2)
| filter total &gt; 10
| sort success_rate asc
| limit 20
```

```
// Execution trend over time
fetch events, from: now() - 7d
| filter event.type == "automation.workflow.execution"
| summarize
    total = count(),
    failed = countIf(execution.status == "FAILED"),
    by:{time_bucket = bin(timestamp, 1h)}
| fieldsAdd failure_rate = round(100.0 * failed / total, decimals: 2)
| sort time_bucket asc
```

```
// Recent failures with error details
fetch events, from: now() - 24h
| filter event.type == "automation.workflow.execution"
| filter execution.status == "FAILED"
| fields timestamp,
         workflow.name,
         execution.id,
         execution.error,
         trigger.type
| sort timestamp desc
| limit 20
```

```
// Task-level performance
fetch events, from: now() - 24h
| filter event.type == "automation.task.execution"
| summarize
    executions = count(),
    failures = countIf(task.status == "FAILED"),
    avg_duration = avg(task.duration),
    max_duration = max(task.duration),
    by:{task.type}
| fieldsAdd failure_rate = round(100.0 * failures / executions, decimals: 2)
| sort executions desc
```

5. Alerting on Workflows

Create a Workflow-Monitoring Workflow

Use a scheduled workflow to monitor other workflows:

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

export default async function({ env }) {
  // Query for failures in the last hour
  const result = await queryExecutionClient.queryExecute({
    body: {
      query: `
        fetch events, from: now() - 1h
        | filter event.type == "automation.workflow.execution"
        | filter execution.status == "FAILED"
        | summarize failures = count(), by:{workflow.name}
        | filter failures &gt;= 3
      `
    }
  });

  const failingWorkflows = result.result.records || [];

  if (failingWorkflows.length &gt; 0) {
    return {
      alert: true,
      failing_workflows: failingWorkflows,
      message: `${failingWorkflows.length} workflows with 3+ failures in the
last hour`
    };
  }

  return { alert: false };
}
```

Conditional Alert Based on Check

```
conditions:
  - name: should_alert
    expression: '{{ result("check_workflows").alert }}'

tasks:
  - name: check_workflows
    type: dynatrace.automations:run-javascript
    # ... check script above

  - name: alert_slack
    type: dynatrace.slack:message
    conditions: [should_alert]
    dependsOn: [check_workflows]
    input:
      channel: "#workflow-alerts"
      message: |
        :warning: *Workflow Health Alert*

        {{ result("check_workflows").message }}

        {% for wf in result("check_workflows").failing_workflows %}
        • {{ wf.workflow_name }}: {{ wf.failures }} failures
        {% endfor %}
```

6. Change Management

Version Control

Export workflows as JSON for version control:

```
# Export via API
curl -X GET "https://&lt;env&gt;/platform/automation/v1/workflows/&lt;id&gt;"
\
  -H "Authorization: Api-Token &lt;token&gt;" \
  -o workflow-backup.json
```

Workflow Naming Convention

```
&lt;team&gt;-&lt;function&gt;-&lt;environment&gt;

Examples:
- sre-problem-notifications-prod
- checkout-auto-remediation-staging
- platform-daily-report-all
```

Testing Changes

1. **Clone workflow** to test version
2. **Modify clone** with changes
3. **Test with On-Demand trigger**
4. **Review execution results**
5. **Promote to production workflow**

Rollback Procedure

1. Disable failing workflow (toggle off)
2. Import previous version from backup
3. Enable restored workflow
4. Verify execution

7. Operational Runbook

Daily Operations

Task	Frequency	Action
Check dashboard	Daily	Review execution success rates
Review failures	Daily	Investigate and fix any failures
Verify connections	Weekly	Test all external connections
Rotate secrets	Monthly	Update API tokens/passwords

Troubleshooting Common Issues

Symptom	Likely Cause	Resolution
Task timeout	External API slow	Increase timeout, add retry
Auth failure	Expired token	Rotate secret
Rate limit	Too many executions	Add throttling, check trigger
Missing data	Query returned empty	Check time range, filters

Escalation Path

```
Level 1: Check execution history, review error message
Level 2: Check connections, verify external services
Level 3: Review workflow logic, check for regressions
Level 4: Contact Dynatrace support
```

Capacity Planning

Limit	Value	Monitor
Max concurrent executions	100	Dashboard query
Max execution time	15 min	Duration metrics
Max tasks per workflow	50	Workflow design
Rate limits	Varies	Rate limit errors

8. Series Summary

What You've Learned

Notebook	Key Topics
WFLOW-01	Workflow fundamentals, components, first workflow
WFLOW-02	Trigger types: Davis problem, metric, schedule, event
WFLOW-03	Slack, Teams, email notification basics
WFLOW-04	Conditional routing, escalation patterns
WFLOW-05	PagerDuty and ServiceNow integration
WFLOW-06	Custom templates, Jinja, Block Kit, Adaptive Cards
WFLOW-07	Auto-remediation, guardrails, approval workflows
WFLOW-08	JavaScript SDK, HTTP requests, custom integrations
WFLOW-09	Security, governance, monitoring, operations

Implementation Checklist

- [ ] Create connections for notification channels
- [ ] Build problem notification workflow
- [ ] Configure severity-based routing
- [ ] Integrate incident management (PagerDuty/ServiceNow)
- [ ] Design custom notification templates
- [ ] Implement auto-remediation (if applicable)
- [ ] Set up workflow monitoring dashboard
- [ ] Configure workflow health alerts
- [ ] Document operational procedures
- [ ] Train team on workflow management

Best Practices Summary

1. **Start simple** - Basic notifications before complex automation

2. **Test thoroughly** - Use On-Demand trigger for testing
3. **Secure by default** - Never hardcode secrets
4. **Monitor everything** - Dashboard for workflow health
5. **Document changes** - Version control and change log
6. **Plan for failure** - Error handling and rollback

Congratulations!

You've completed the WFLOW series. You now have the knowledge to:

- Build event-driven notification workflows
- Integrate with incident management platforms
- Create custom automation with JavaScript
- Implement auto-remediation safely
- Operate workflows in production

References

- Dynatrace Workflows Documentation
- Workflow Actions Reference
- Dynatrace SDK
- IAM Permissions
